Graphs Meeting LLMs

Introduction

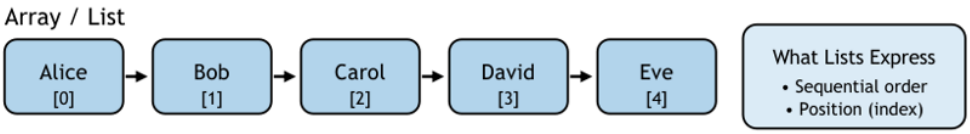
The convergence of Large Language Models (LLMs) and Graph technologies represents one of the most promising frontiers in artificial intelligence. While LLMs excel at understanding and generating human language, they often struggle with structured reasoning, factual accuracy, and explainable outputs.

On the other hand, Knowledge Graphs, provide structured, interconnected data that captures real-world relationships with precision.

Understanding Graphs

Linear structures: Lists and Arrays

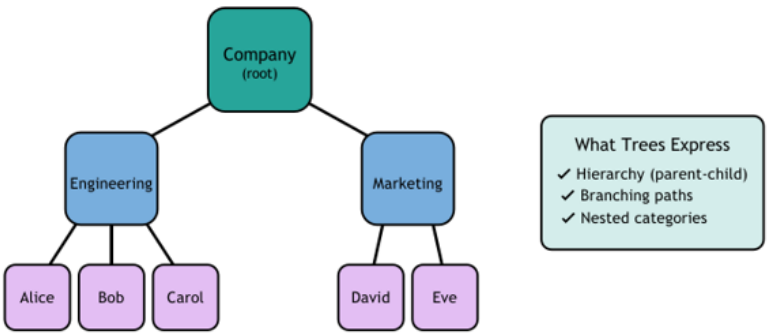
The simplest way to organize data is in a sequence.



In a linear structure, you might store them alphabetically or by employee ID. But this tells you nothing about who manages whom, who works together on projects, or who has collaborated before.

Adding Hierarchy: Trees

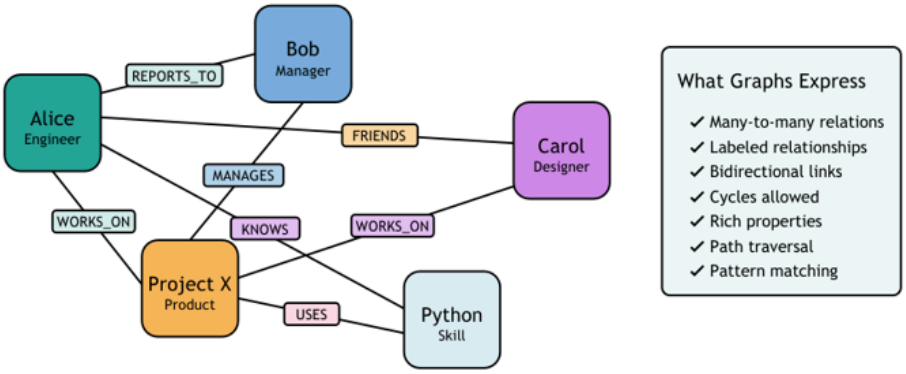
Trees improve on lists by introducing hierarchy. Instead of a single chain, a tree allows one "parent" node to connect to multiple "children." This is perfect for representing organizational charts, file systems, or taxonomies.



Trees add hierarchy with parent-child relationships, but each node can still only have one parent

The Power of Graphs: Any-to-Any Connections

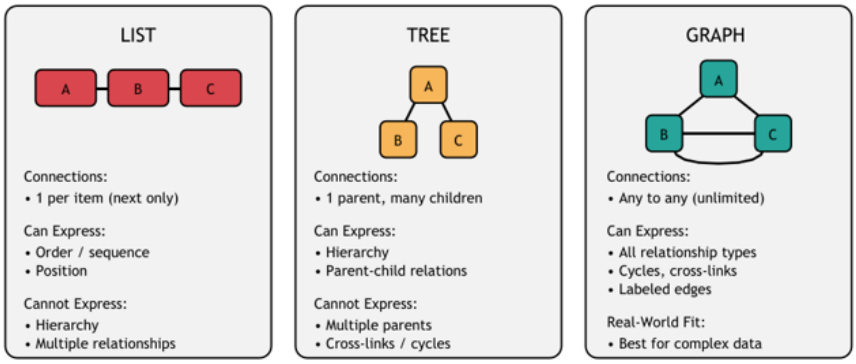
Graphs remove the linearity and hierarchy constraints entirely. In a graph, any node can connect to any other node, or even to itself. There's no limit on the number of connections, and crucially, each connection (called an "edge") can have a label that describes the relationship type.



Graph Terminology: Nodes, Edges, and Properties

Understanding graphs requires just a few key terms. A graph consists of nodes (also called vertices), which represent entities like people, places, or concepts. Nodes connect to each other via edges (also called relationships), which represent how entities relate.

Both nodes and edges can have properties, key-value pairs that store additional information. Like the edge *WORKS_AT* might have properties for start_date and role.



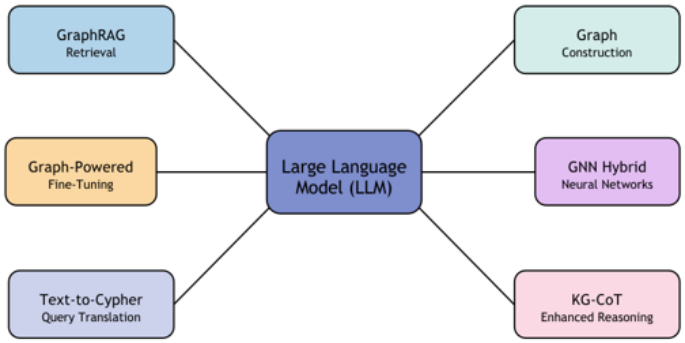
When someone says "knowledge graph," they typically mean a graph where nodes represent real-world entities and edges represent factual relationships between them, essentially a structured database of knowledge.

Why Graphs Matter for LLMs

LLMs excel at understanding and generating human language, but they process text as sequences

They don't inherently understand that "Apple" the company is connected to "Tim Cook" who is connected to "Stanford University." Instead it relies on patterns learned during training rather than explicit relationship traversal.

Graphs provide exactly what LLMs lack: explicit, structured representations of how entities relate to each other. By combining graph structures with language models, we can give LLMs access to verified facts and relationships, enable multi-step reasoning along explicit paths.



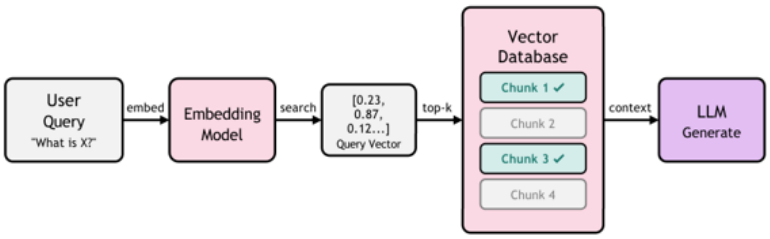
GraphRAG: Graph Retrieval-Augmented Generation

GraphRAG enhances traditional RAG by replacing simple vector similarity search with structured knowledge graph traversal, enabling multi-hop reasoning and more accurate, contextually grounded responses.

Traditional RAG System

Traditional Retrieval-Augmented Generation relies on vector similarity to find relevant documents.

1. Embed the user query into a high-dimensional vector.
2. Embed the document chunks into the same vector space.
3. Compute similarity (e.g., cosine) between the query vector and chunk vectors.
4. Retrieve the top-k most similar chunks as context for generation.



Note: This method finds semantic matches but does not model relationships across facts, so multi-hop questions may miss required context.

Traditional RAG Challenges

Retrieves semantically similar but potentially disconnected chunks. Lacks understanding of entity relationships. Cannot traverse connections to find related information. Limited to single-hop retrieval. Twitter → Acquisition → Company → CEO.

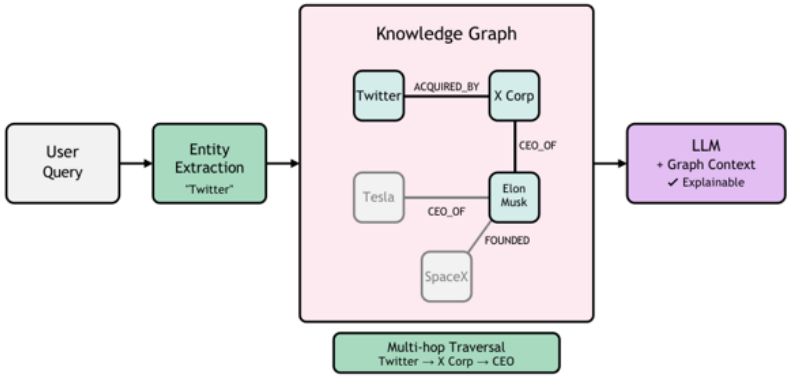
What GraphRAG Solves

Enables multi-hop reasoning through relationship traversal. Preserves entity connections and context. Provides explainable retrieval paths. Combines structured knowledge with semantic search.

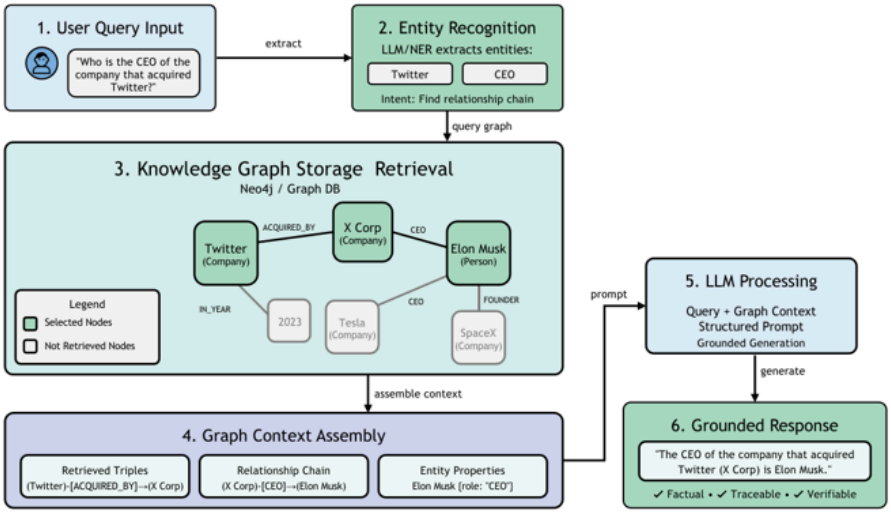
GraphRAG Architecture

GraphRAG replaces or augments the vector store with a knowledge graph, enabling retrieval through relationship traversal. The system can follow edges between entities to gather contextually relevant information across multiple hops.

1. Identify key entities in the user query.
2. Locate those entities in the knowledge graph.
3. Traverse the surrounding graph neighborhood (follow connected edges).
4. Collect related nodes, relationships, and relevant properties as context.
5. Assemble the retrieved subgraph into a rich context for generation.



A more detailed view of the system is as follows:


1. User Query Input
"Who is the CEO of the company that acquired Twitter?"
extract
2. Entity Recognition
LLM/NER extracts entities:
Twitter
CEO
Intent: Find relationship chain
query graph
3. Knowledge Graph Storage Retrieval
Neo4j / Graph DB
Twitter (Company)
ACQUIRED_BY
X Corp (Company)
CEO
Elon Musk (Person)
IN_YEAR
2023
Tesla (Company)
CEO
FOUNDER
SpaceX (Company)
Legend
Selected Nodes
Not Retrieved Nodes
assemble context
4. Graph Context Assembly
Retrieved Triples
(Twitter)-[ACQUIRED_BY]→(X Corp)
Relationship Chain
(X Corp)-[CEO]→(Elon Musk)
Entity Properties
Elon Musk [role: "CEO"]
prompt
5. LLM Processing
Query + Graph Context
Structured Prompt
Grounded Generation
generate
6. Grounded Response
"The CEO of the company that acquired Twitter (X Corp) is Elon Musk."
✓ Factual • ✓ Traceable • ✓ Verifiable